ОТЧЁТ ПО ДОМАШНЕМУ ЗАДАНИЮ № 6

Отчет по Д36

Цель

Настроить GitHub Actions для автоматической проверки и развертывания Job Platform на удаленный сервер при обновлении `main`.

Trigger

Workflow расположен в корневом `.github/workflows/deploy-hw6.yml`, поэтому GitHub распознает его. Автоматический trigger - push в ветку `main` с изменениями Д36 или workflow. Для повторной проверки добавлен `workflow_dispatch`.

CI

Матрица из семи jobs проверяет каждый микросервис:

1. `npm ci` устанавливает версии из lock-файла.
2. `npm run build` компилирует TypeScript.
3. `docker build` проверяет Dockerfile.

Отдельный job валидирует `docker-compose.yml`. Deploy job зависит от обеих проверок и не запускается при ошибке.

CD

Deploy job:

1. подключается к Ubuntu-серверу по SSH-ключу;
2. клонирует репозиторий или обновляет checkout до `origin/main`;
3. создает `.env` из GitHub Actions Secrets;
4. валидирует Compose;
5. собирает образы последовательно из-за ограниченной RAM;
6. запускает Docker Compose и миграции;
7. проверяет dictionary-service локально и через Nginx;
8. выводит итоговый `docker compose ps`.

Secrets

Для SSH используются `SSH_HOST`, `SSH_USER`, `SSH_PRIVATE_KEY`. Runtime-конфигурация передается через `JWT_SECRET_KEY`, `SERVICE_TOKEN`, `DB_PASSWORD`, `RABBITMQ_USER` и `RABBITMQ_PASSWORD`. В репозитории нет их значений.

Проверка

Итоговый GitHub Actions run `36343708764` для commit `e541c595c8d546be7843347cf4414d9623e530f4` завершился успешно. Все семь jobs микросервисов, проверка Compose и deploy имеют статус `success`.

После автодеплоя независимо проверено:

- checkout сервера указывает на тот же commit;
- работают 7 backend-сервисов и 7 healthy PostgreSQL;
- RabbitMQ имеет статус `healthy`;
- порты 3001-3007 привязаны только к `127.0.0.1`;
- PostgreSQL и RabbitMQ не публикуют порты на host;
- `https://185.236.24.201/job-platform/api/v1/industries` возвращает HTTP 200;
- internal API возвращает HTTP 404.

Ссылка на run: <https://github.com/frizyyu/ITMO-ACS-Backend-2026-A/actions/runs/36343708764>.

Первый холодный deploy выявил два ограничения маломощного сервера: последовательная сборка заняла больше 20 минут, а RabbitMQ под пиковой нагрузкой не успел инициализироваться в прежнее окно healthcheck. Поэтому timeout SSH-команды увеличен до 40 минут, а RabbitMQ получил `start_period: 180s`. Проверка здоровья сохранена и итоговый запуск подтвердил исправления.

Вывод

Процесс от push в `main` до обновления приложения на удаленном сервере автоматизирован GitHub Actions. Деплой выполняется только после успешных CI-проверок.